

Experiment - 13

AIM:

Implement your own ping program

Algorithm:

1. Create a UDP socket
2. Set a timeout of 2 seconds
3. Record start time
4. Send the message "ping" to server
5. Wait to receive a response from the server
 - * If received, record end time and display the reply with round-trip time
 - * If timeout occurs, print "request time out"
6. Close the socket.

Input:

import socket

import time

def ping_server(host = '127.0.0.1', port = 12345):
 with

socket.socket(socket.AF_INET, socket.SOCK_DGRAM) as s:

try:

s.settimeout(2)

start = time()

```
s_sendto (b'ping', (host, port))  
data, addr = s.recvfrom (1024)  
end = time.time()
```

```
print(f"Received {data.decode()} from {addr}")  
{addr} in {end - start} seconds
```

```
except socket.timeout:
```

```
print("Request timed out")
```

```
if __name__ == "__main__":
```

```
ping_server()
```

Output:

Received ping from ('127.0.0.1', 12345) in 0.00
seconds request timed out.

Result:

Successfully implemented chat client server using
tcp/ip sockets.

27/12/23

b) Algorithm:

- 1) Create a udp socket
- 2) Bind the socket to server IP and port
- 3) continuously wait for Incoming Messages
- 4) when a message is received:
 - * Displayed the message and client address
 - * send back "ping" to the client.
- 5) Repeat step 3 indefinitely.

Input:

```
import socket
def start_server(host = '127.0.0.1', port = 4345):
    with
    socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
    as s:
        s.bind((host, port))
        print(f"UDP server running on {host}:{port}")
        while True:
            data, addr = s.recvfrom(1024)
            print(f"Received message from {addr}: {data.decode()}")
            s.sendto(b'pong', addr)
if __name__ == "__main__":
    start_server()
```


07/10

UDP Server running on 127.0.0.1 : 12345
Received message from ('127.0.0.1' 52345) Ping

✓
2010

Result:

ping program is implemented successfully